

LEARNING RESOURCE GUIDE

Generative AI for Beginners | Professional Participant Workbook

Section	Section 3: Large Language Models (LLMs)
Lecture	Lecture 3.6: The Generation Process

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- Describe the autoregressive generation process by which LLMs produce text one token at a time.
- Explain what a probability distribution over the vocabulary is and how the model uses it at each generation step.
- Distinguish between greedy decoding, beam search, temperature sampling, top-k sampling, and top-p (nucleus) sampling, and when to use each.
- Explain the role of stopping criteria and how they control the length and boundaries of generated outputs.
- Articulate the difference between inference (generation) and training, and why inference is less computationally intensive.

Introduction

When you press send on a message to ChatGPT, the model does not instantly retrieve a pre-stored answer. It generates the response token by token, each token selected probabilistically from a distribution over the entire vocabulary. This generation process — called autoregressive sampling — is how all modern large language models produce text. Understanding it illuminates why LLMs have the properties they do: why longer responses take more time, why the same prompt can produce different responses each time, why you can control creativity versus consistency, and why models sometimes produce plausible-sounding but incorrect continuations.

The generation process is the synthesis of everything the model has learned during training. The model's parameters — billions of numerical weights — have encoded patterns from the training corpus. At each generation step, these parameters transform the current context into a probability distribution over the vocabulary. A high probability for 'Paris' following 'The capital of France is' reflects the model having seen this pattern countless times during training. A high probability for the wrong answer to a rare question reflects the same mechanism operating on less reliable signal.

Controlling the generation process through sampling parameters — temperature, top-k, top-p — is a practical skill for anyone building LLM-powered applications. These parameters determine whether the model behaves deterministically or creatively, and they explain many of the practical behaviours users observe in generation-based AI systems. Getting these parameters right for a specific application is often as important as selecting the model itself.

Core Concepts Explained

Autoregressive Generation: One Token at a Time

Definition: Autoregressive generation is the process by which LLMs produce output. At each step, the model takes the entire sequence produced so far (the original prompt plus all tokens generated up to that point) as input, and outputs a probability distribution over all tokens in its vocabulary. One token is selected from this distribution, appended to the sequence, and the process repeats until a stopping criterion is met.

Business relevance: The autoregressive property explains several important practical behaviours. Response generation time scales linearly with output length — each additional token requires a full forward pass through the model. It also explains why token streaming (displaying text as it is generated rather than waiting for completion) dramatically improves user experience for long responses.

Practitioner relevance: For application developers, autoregressive generation means latency depends on output length, not just input length. Prompts that elicit long responses will have higher latency and cost than prompts that elicit short responses, even for identical input lengths. Designing prompts to elicit appropriately scoped responses is a latency and cost optimisation.

Real-world example: When generating the response to 'Translate to French: The cat sat on the mat', the model first generates 'Le' (highest probability), then given 'Le' it generates 'chat' (highest probability given 'Le'), then 'est' or 'assis', and so on. Each token is generated based on the full context including all previously generated tokens, not just the most recent one.

Risks or limitations: Because each generated token is conditioned on all previous tokens, errors compound. If the model generates an incorrect token early in a response — committing to a wrong fact, a wrong name, or a wrong logical step — all subsequent tokens are generated conditioned on that error, often producing a coherent but incorrect continuation. This is why LLM errors are often subtly wrong rather than obviously nonsensical.

Sampling Strategies: From Greedy to Creative

Definition: At each generation step, the model outputs a probability distribution over the vocabulary. How a token is selected from this distribution is determined by the decoding strategy. Greedy decoding always selects the highest-probability token. Beam search maintains multiple candidate sequences simultaneously. Temperature sampling scales the distribution before sampling; top-k and top-p sampling restrict which tokens are candidates for selection.

Business relevance: Different applications require different points on the determinism-creativity spectrum. Customer service chatbots benefit from low-temperature, deterministic responses (consistent, professional, predictable). Creative writing tools benefit from higher temperature (varied, inventive responses). Code generation typically uses low temperature (correct code is usually deterministic) with some diversity for suggestions.

Practitioner relevance: Temperature, top-k, and top-p are the primary levers for controlling generation behaviour in production API calls. Understanding what each does enables informed configuration decisions. Leaving these parameters at defaults may not be appropriate for your application — the defaults are tuned for general chat use, not for your specific task.

Real-world example: At temperature 0.0, generating 'The capital of France is' will always produce 'Paris'. At temperature 1.5, the model might generate 'Paris', 'Lyon', 'London', or even nonsensical continuations because high temperature flattens the probability distribution, giving low-probability tokens a meaningful chance of

selection. For factual Q&A, temperature near 0 is appropriate; for brainstorming, higher temperature produces more varied ideas.

Risks or limitations: High temperature increases creativity but also increases the probability of generating factually incorrect, grammatically poor, or logically incoherent responses. Temperature should be calibrated to the application's tolerance for creative variation versus requirement for accuracy. Setting temperature too low can cause repetitive, loop-prone generation in some models.

Temperature, Top-k, and Top-p (Nucleus) Sampling

Definition: Temperature divides the model's raw output logits before converting to probabilities — lower temperature (< 1.0) makes the distribution peakier (more deterministic), higher temperature (> 1.0) makes it flatter (more random). Top-k sampling restricts selection to the k highest-probability tokens, discarding the rest. Top-p (nucleus) sampling selects the smallest set of tokens whose cumulative probability exceeds p, then samples from this set — adapting the candidate set size to the distribution's shape.

Business relevance: Top-p sampling is generally preferred over top-k in production because it adapts dynamically to the distribution's shape. When the model is confident (one token dominates the distribution), top-p selects from a small nucleus. When the model is uncertain (many tokens have similar probability), top-p samples from a larger set. This prevents both overly repetitive responses (too-low top-k) and nonsensical responses (too-high top-k when distribution is peaky).

Practitioner relevance: For production applications, a common starting configuration is temperature 0.2 to 0.7 with top-p 0.9 to 0.95. Experiment with these parameters on representative test prompts and evaluate both response quality and diversity. Document the chosen values in your application configuration as they affect reproducibility.

Real-world example: If the model's next-token distribution has 80 percent probability on 'Paris' and 15 percent on 'Lyon', top-p 0.95 would select from a nucleus of {Paris, Lyon} covering 95 percent of the probability. If the distribution is more uniform with the top 20 tokens each having ~5 percent probability, top-p 0.95 would sample from a much larger nucleus — automatically adapting to the distribution's entropy.

Stopping Criteria and Inference vs Training

Definition: Stopping criteria determine when the model stops generating. The primary stopping criterion is the end-of-sequence (EOS) special token — the model learns during training to generate this token at natural response boundaries. Additional stopping criteria include: maximum token length (to control costs and prevent runaway generation), stop sequences (specific strings that halt generation), and API-level timeout limits.

Business relevance: Stop sequences are a practical tool for structured output generation. If you want the model to generate a JSON object and stop, you can define '}' as a stop sequence. If you are generating responses in a templated format with clear section markers, stop sequences allow the model to fill in a structured slot without generating beyond the expected boundary.

Practitioner relevance: Understanding that inference (generation) is computationally lighter than training is relevant for architecture decisions. Training requires storing gradients and running backpropagation across billions of parameters — extremely memory- and compute-intensive. Inference requires only a forward pass. This is why fine-tuning and training are done on GPU clusters while inference can run on smaller hardware, and why inference latency is measured in seconds rather than the hours or days required for training runs.

Real-world example: A customer support application generates responses in the format 'Resolution: [text] | Confidence: [Low/Medium/High]'. By setting '|' as a stop sequence, the application can run two separate generation calls — one for the resolution, one for the confidence level — with the model stopping cleanly at the delimiter each time, enabling post-processing of the structured output.

Practical Applications

Calibrating Generation Parameters for Production Applications

Every production LLM application should define explicit temperature, top-p, and max-token values appropriate for the use case. For factual question-answering or code generation, low temperature (0.0 to 0.3) minimises variance and hallucination. For creative generation, brainstorming, or diverse suggestions, higher temperature (0.7 to 1.0) produces more varied outputs. Document these parameters alongside the system prompt as part of the application specification.

Streaming for User Experience

Because autoregressive generation produces tokens sequentially, streaming APIs allow applications to display output as it is generated rather than waiting for completion. This dramatically reduces perceived latency — users begin reading a response that is still being generated. Most production-facing LLM applications implement streaming for any response expected to be longer than a few sentences. Streaming also enables early stopping by the application if the user interacts before generation completes.

Structured Output Generation with Stop Sequences

For applications that require structured or machine-parseable outputs (JSON, XML, CSV, templated text), combining precise prompting with stop sequences and format instructions enables reliable structured generation. Modern LLMs also support JSON mode or structured output APIs that constrain the generation to valid JSON schema. These approaches reduce parsing complexity and improve reliability compared to parsing free-form text.

Best Practices

- Set `max_tokens` explicitly for every API call. Uncontrolled generation can produce unexpectedly long and expensive responses. Set a `max_tokens` limit appropriate to the expected response length for each endpoint in your application.
- Use stop sequences for structured or bounded outputs. Whenever your application expects a response in a specific format or bounded by a delimiter, define stop sequences. This is more reliable than asking the model to stop at a specific point in a prompt.
- Test generation at multiple temperature settings before deploying. The optimal temperature is application-specific. Test your prompts at temperatures ranging from 0.0 to 1.0 and evaluate both quality and variance before selecting a production value.
- Implement retry logic for generation failures. API calls can fail due to rate limits, timeouts, or service errors. Implement exponential backoff retry logic and define fallback behaviour for generation failures in production applications.

Common Mistakes and Misconceptions

Misconception: Leaving temperature at default for all use cases.

Correct approach: Default temperature is tuned for general chat — it is not appropriate for all applications. Factual retrieval benefits from lower temperature; creative generation benefits from higher temperature. Calibrate temperature explicitly for each application type.

Misconception: Not setting a max_tokens limit in production.

Correct approach: Without a max_tokens limit, the model may generate very long responses, increasing costs and latency. Always set a max_tokens value appropriate to the expected response length for each API endpoint.

Misconception: Expecting the same generation results every time at temperature > 0.

Correct approach: Any temperature above 0 introduces randomness — the same prompt will produce different responses in different calls. If your application requires reproducible outputs, use temperature 0 (or as close to 0 as the API supports) and be aware that even then, different model deployments may produce slightly different outputs.

Decision Framework: Generation Parameter Selection Guide

Use this framework to select appropriate generation parameters for different application types.

Application Type	Temperature	Top-p	Max Tokens	Stop Sequences
Factual Q&A	0.0–0.2	0.9	Match expected answer length	None usually needed
Code generation	0.1–0.3	0.95	Task-appropriate limit	Language-specific delimiters
Customer service chatbot	0.3–0.5	0.9	200–500 typical	None or format markers
Brainstorming / ideation	0.7–1.0	0.95	Task-appropriate	Separator tokens
Creative writing	0.8–1.2	0.95–1.0	Chapter/section length	Section markers
Data extraction / parsing	0.0–0.1	0.9	Structured output length	JSON delimiters (})
Summarisation	0.2–0.4	0.9	25–50% of input length	None

How to use this: These are starting points — always benchmark on representative examples from your specific application before finalising production values.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

1. A developer notices that their LLM-based customer service chatbot gives slightly different answers to the same question each time, sometimes including incorrect information. Which parameter change is most likely to improve consistency and accuracy?

- A. Increase temperature to 1.5 to encourage more creative responses.
- B. Reduce temperature to 0.1 to 0.3 to make the distribution more deterministic and reduce random sampling of low-probability (potentially incorrect) tokens.
- C. Increase max_tokens to allow the model to provide more complete answers.
- D. Add a stop sequence to limit response length.

Answer: B. Reduce temperature to 0.1 to 0.3 to make the distribution more deterministic and reduce random sampling of low-probability (potentially incorrect) tokens.

Why: Higher temperature flattens the probability distribution, increasing the chance that low-probability tokens (including incorrect ones) are selected. Reducing temperature makes the model more consistently choose the highest-probability continuation.

2. An application generates responses and then stops when it encounters the string '###'. After adding this stop sequence to the API call, the model occasionally generates past '###' anyway. What is the most likely cause?

- A. Stop sequences are not supported by the API being used.
- B. The string '###' might be tokenized as multiple tokens, and the stop sequence matching must align with token boundaries, not character boundaries.
- C. Temperature is set too high, overriding the stop sequence.
- D. The stop sequence needs to be placed in the system prompt instead.

Answer: B. The string '###' might be tokenized as multiple tokens, and the stop sequence matching must align with token boundaries, not character boundaries.

Why: *Stop sequences work at the token level. If the stop sequence string spans multiple tokens or if the model generates token sequences that spell the string but don't align with how the tokenizer splits it, matching may fail. Testing stop sequences with the actual tokenizer is important.*

3. Why does response generation time for an LLM scale with output length rather than input length?

- A. Because the model reads the input multiple times to understand it fully.
- B. Because autoregressive generation requires a separate full forward pass through the model for each output token generated, while the input is processed in a single pass.
- C. Because longer outputs require more memory, which slows the GPU.
- D. Because stop sequences must be checked after each token, adding overhead proportional to output length.

Answer: B. Because autoregressive generation requires a separate full forward pass through the model for each output token generated, while the input is processed in a single pass.

Why: *The input (prompt) is processed in one forward pass (though KV caching optimises this). Each output token requires its own forward pass to compute the next-token distribution. Therefore, latency scales linearly with the number of generated tokens.*

Reflection Questions

1. Autoregressive generation produces text by repeatedly sampling the next token — which means all LLM output is, at a fundamental level, a statistical continuation of the input. What are the implications of this for applications that require guaranteed correctness, such as medical advice or legal analysis?
2. Temperature is often described as controlling 'creativity'. But creativity in humans involves novel combinations of ideas driven by experience and insight, not just randomness. To what extent is temperature-based 'creativity' in LLMs similar to or different from human creativity?
3. The same prompt at the same temperature can produce different outputs on different runs. How would you design an experiment to evaluate LLM output quality given this inherent non-determinism?
4. Stop sequences, max_tokens limits, and explicit format instructions are mechanisms for constraining LLM output to fit application requirements. What does the need for these constraints tell you about the nature of LLMs as software components compared to traditional deterministic software functions?

Key Takeaways

- Autoregressive generation produces text one token at a time, with each token selected from a probability distribution conditioned on the full preceding context.
- Temperature controls the 'peakiness' of the probability distribution: low temperature (near 0) is more deterministic; high temperature (above 1) is more random and creative.
- Top-p (nucleus) sampling dynamically selects the smallest set of tokens covering a target cumulative probability, adapting to the distribution's confidence level.
- Stop sequences and max_tokens limits are essential production controls that prevent runaway generation and enable structured output formats.
- Inference (generation) is a forward pass only — far less computationally intensive than training, which requires backpropagation and gradient storage.
- Generation latency scales with output length because each token requires its own forward pass; streaming output reduces perceived latency for users.
- Production generation parameters (temperature, top-p, max_tokens, stop sequences) should be explicitly calibrated for each application type — defaults are designed for general chat, not specific tasks.

Recommended Next Actions

5. Experiment with temperature: send the same prompt to a GPT interface or API three times at temperature 0 and three times at temperature 1.0. Compare the responses for consistency and creativity differences.
6. Try setting a stop sequence in a playground or API call. Use a format like 'Answer: [X] | Done' and set '|' as your stop sequence. Observe whether the model stops cleanly at the delimiter.
7. Design a generation parameter specification for a hypothetical application in your domain (factual Q&A, creative writing, data extraction). Justify your temperature, top-p, max_tokens, and stop sequence choices.
8. Add to your glossary: Autoregressive Generation, Temperature, Top-k Sampling, Top-p Sampling, Nucleus Sampling, Stop Sequence, Greedy Decoding, Beam Search, EOS Token.